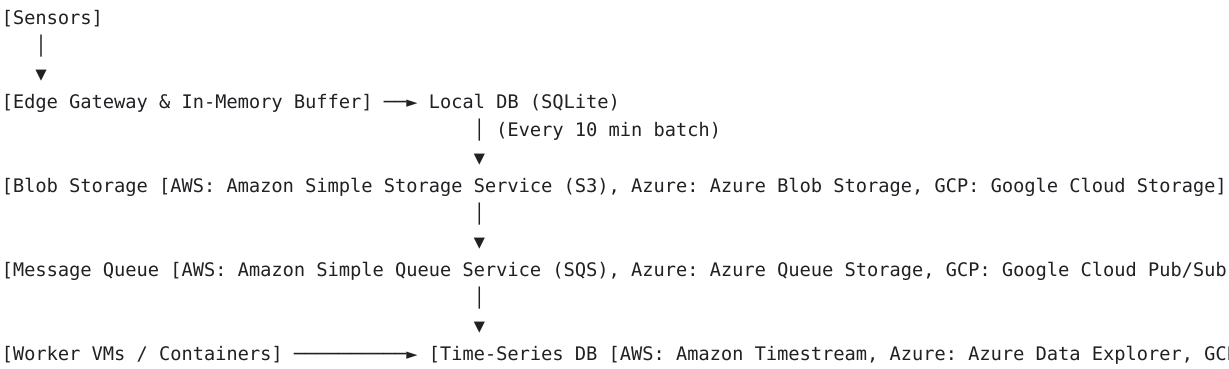


System Design Masterclass: Industry Scenarios & Architectures

1. Smart Home Automation (IoT Edge & Batching)

The Problem: Streaming millions of sensor pings per second directly to the cloud is astronomically expensive and fails when the internet connection drops. **The Solution:** Use Edge computing with a bounded buffer. Process real-time triggers locally, and batch historical data for the cloud.

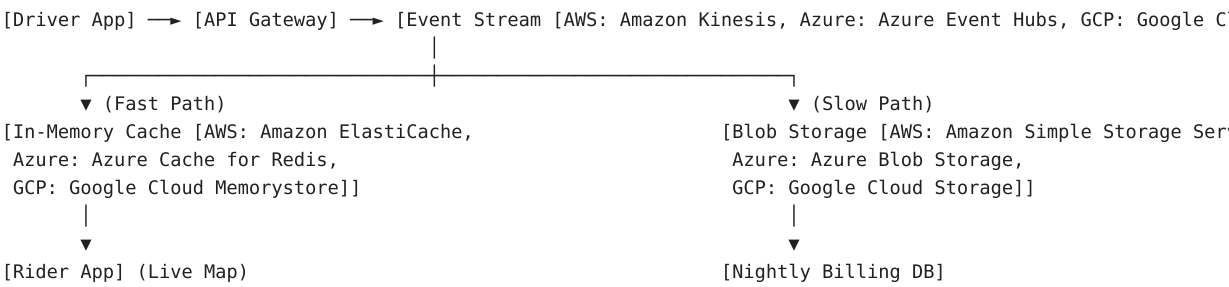
Architecture Diagram:



2. Ride-Hailing GPS Tracking (Uber/Ola)

The Problem: 100,000 drivers sending GPS pings every 5 seconds will crash standard REST APIs and relational databases. **The Solution:** Use high-throughput event streaming for ingestion, a geospatial in-memory cache for live maps, and batch storage for billing.

Architecture Diagram:



3. High-Concurrency Ticket Booking (BookMyShow)

The Problem: 1 million users trying to buy 50,000 seats simultaneously will crash servers (Thundering Herd) and cause double-bookings. **The Solution:** Block 95% of traffic at the CDN (Waiting Room) and use single-threaded distributed locks with time-to-live (TTL).

Architecture Diagram:

```

[1M Users]
|
▼
[CDN / Edge Network [AWS: Amazon CloudFront, Azure: Azure Front Door, GCP: Cloud CDN]] (Waiting Room)
| (Allows 50k users)
▼
[Web Servers] → [In-Memory Cache [AWS: Amazon ElastiCache, Azure: Azure Cache for Redis, GCP: Google Cloud M
| (Uses SETNX for 5-minute Seat Locks)
▼ (On Payment Success)
[Relational DB [AWS: Amazon Aurora (PostgreSQL), Azure: Azure Database for PostgreSQL, GCP: Cloud SQL]]

```

4. E-Commerce Search (Amazon)

The Problem: Searching 100 million products with typos and filters using SQL LIKE takes 10+ seconds and crashes the database. **The Solution:** Use an Inverted Index (Search Engine) and sync data asynchronously using Change Data Capture (CDC).

Architecture Diagram:

```

[Primary Relational DB] → [CDC Tool (Debezium)] → [Event Stream [AWS: Amazon Managed Streaming for Apache
|
▼
[User Search Query] → [Full-Text Search Engine [AWS: Amazon OpenSearch Service]

```

5. Social Media News Feed (Twitter)

The Problem: Pushing a celebrity's tweet (100M followers) to individual feeds crashes the write database. Pulling everyone's tweets crashes the read database. **The Solution:** Hybrid Fan-out. Push normal user tweets; pull celebrity tweets on the fly.

Architecture Diagram:

```

[Tweet] → Is Celebrity?
    ├── YES → [Global Cache [AWS: Amazon ElastiCache, Azure: Azure Cache for Redis, GCP: Cloud Memor
    └── NO  → [Message Queue] → Push to Follower's [Personal In-Memory Feed Cache]

[App Opens] → Merges [Personal Feed] + [Celebrity Cache] in memory < 100ms.

```

6. Digital Wallet Transactions (FinTech / Saga Pattern)

The Problem: Transferring money across distributed microservices risks losing money if the network fails midway. **The Solution:** Use an Orchestrator (Saga Pattern) with compensating transactions (refunds) and Idempotency Keys to prevent double-charging.

Architecture Diagram:

```

[Transaction Orchestrator]
├── 1. Deduct $50 → [Wallet DB A] (Success)
├── 2. Add $50    → [Wallet DB B] (Fails!)
└── 3. Compensation → [Wallet DB A] (Refund $50 automatically)

```

7. Distributed API Rate Limiter

The Problem: Hackers sending 10,000 requests/sec across 50 load-balanced servers to cause a DDoS. **The Solution:** Use the Token Bucket algorithm stored in a centralized, ultra-fast in-memory cache.

Architecture Diagram:

[API Servers (x50)] → [In-Memory Cache [AWS: Amazon ElastiCache, Azure: Azure Cache for Redis, GCP: Cloud Memorystore] (Stores: { "user_123_tokens": 99 })] → Rejects if 0.

8. Video Streaming (YouTube)

The Problem: Serving large 5GB video files directly from one server causes buffering and maxes out bandwidth. **The Solution:** Transcode video into multiple resolutions, slice into 10-second chunks, and cache globally.

Architecture Diagram:

[Raw Video] → [Transcoder / Media Service [AWS: AWS Elemental MediaConvert, Azure: Azure Media Services, GCP: Cloud Video Intelligence] (Generates 10s chunks)]
↓
[CDN / Edge Network [AWS: Amazon CloudFront, Azure: Azure Front Door, GCP: Cloud CDN]] → [User Device]

9. Real-Time Chat (WhatsApp)

The Problem: HTTP polling crashes servers. Users are connected to different physical servers. **The Solution:** Persistent WebSockets and a Pub/Sub routing bus.

Architecture Diagram:

[User A] ←(WebSocket)→ [Chat Server 1]
|
[Pub/Sub Message Bus [AWS: Amazon SNS/ElastiCache, Azure: Azure Service Bus, GCP: Cloud Pub/Sub]]
|
[User B] ←(WebSocket)→ [Chat Server 2]

10. Enterprise RAG (Secure AI Chatbot)

The Problem: The LLM hallucinates, cannot fit 1M documents in context, and might leak CEO secrets to interns. **The Solution:** Chunking, Embeddings, and Vector Databases with strict Metadata Filtering.

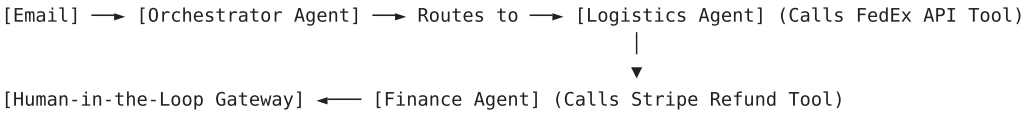
Architecture Diagram:

[User Query] → [Embedding Model] → [Vector DB [AWS: Amazon OpenSearch Serverless, Azure: Azure Cosmos DB for NoSQL, GCP: Cloud Firestore] (Filters by user_clearance_level) (Returns 5 safe paragraphs)]
↓
[LLM [AWS: Amazon Bedrock, Azure: Azure OpenAI, GCP: Vertex AI]]

11. Autonomous AI Agents (Customer Support)

The Problem: LLMs are text generators and cannot execute real-world actions safely. **The Solution:** Tool Use (Function Calling), Multi-Agent orchestration, and Human-in-the-Loop gateways.

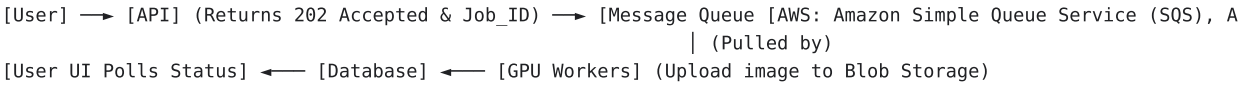
Architecture Diagram:



12. AI Image Generation (Async Queue)

The Problem: Generating images takes 15 seconds. Standard HTTP APIs time out. You have 50k requests but only 500 GPUs. **The Solution:** Asynchronous API, message queues, and worker pulling.

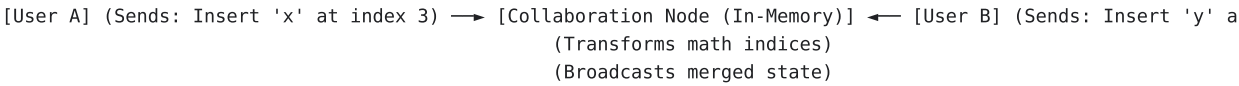
Architecture Diagram:



13. Collaborative Editor (Google Docs)

The Problem: Database locks prevent live typing. Network latency causes screens to go out of sync. **The Solution:** Operational Transformation (OT) or CRDTs via WebSockets.

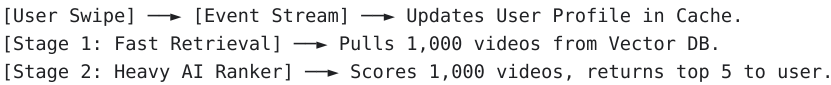
Architecture Diagram:



14. Video Recommendation (TikTok)

The Problem: Scoring 50 million videos with a Deep Neural Network for every swipe takes too long. **The Solution:** A two-stage funnel (Candidate Generation + Heavy Ranking) updated by real-time stream processing.

Architecture Diagram:



15. Long-Running AI Companion (Context Window Management)

The Problem: An LLM has a strict context window limit (e.g., 128k tokens). Passing a user's entire 6-month chat history for every message is too expensive, too slow, and will cause the LLM to crash or drop information. **The Solution:** A Three-Tiered Memory Architecture. We use a Sliding Window for short-term flow, an automated background summarizer for mid-term context, and a Vector Database for long-term semantic retrieval.

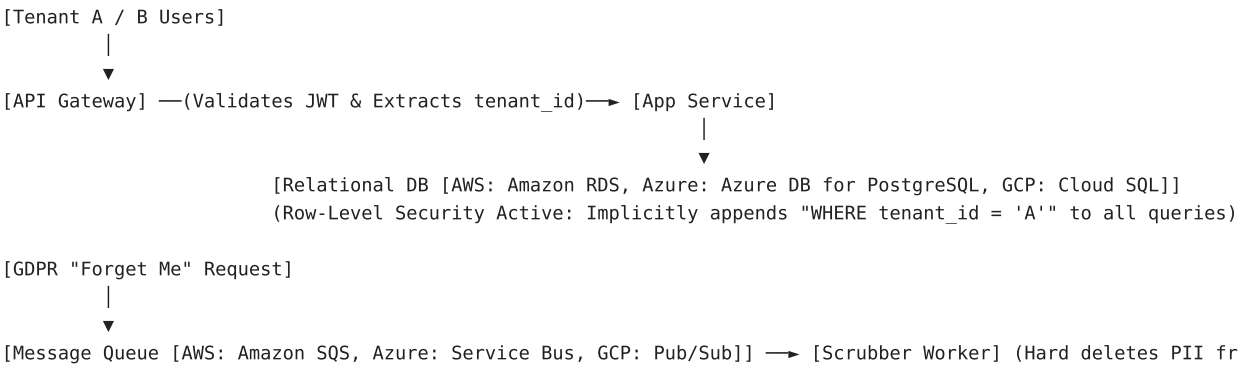
Architecture Diagram:



16. Multi-Tenant B2B SaaS (GDPR & Data Isolation)

The Problem: Hosting multiple companies (tenants) in the same database table risks catastrophic data leakage (Tenant A seeing Tenant B's data). Furthermore, GDPR requires the strict ability to completely erase a user's data (Right to be Forgotten) across the entire system. **The Solution:** Implement Row-Level Security (RLS) in the database to guarantee tenant isolation at the query level, combined with an automated async worker pattern to scrub Personally Identifiable Information (PII) across all systems.

Architecture Diagram:

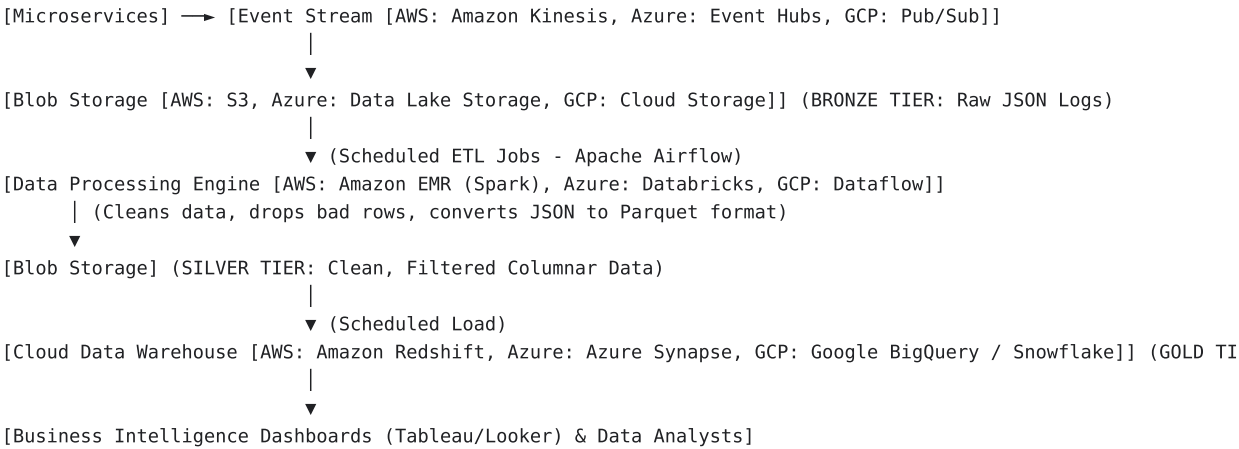


17. Enterprise Data Lakehouse (Medallion Architecture ETL/ELT)

The Problem: A company has 50 different microservices generating 10 Terabytes of messy, unstructured JSON logs every day. Data analysts and ML engineers need to run fast, complex SQL queries on this data, but querying raw JSON across operational databases is impossibly slow and ruins

live application performance. **The Solution:** Build a Data Pipeline using the Medallion Architecture (Bronze, Silver, Gold). Extract data asynchronously, store raw records cheaply in a Data Lake, transform them into highly optimized columnar formats (like Parquet), and load them into a Cloud Data Warehouse.

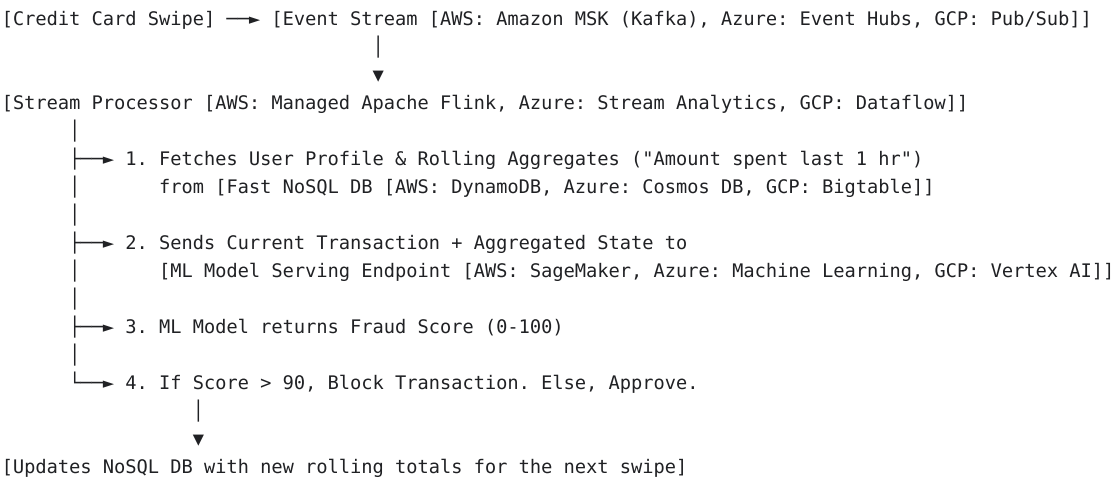
Architecture Diagram:



18. Real-Time Fraud Detection (Stream Processing)

The Problem: When a user swipes a credit card, the transaction must be approved or denied in under 50 milliseconds. The AI model needs to know if the user is suddenly swiping in a new country, or if they've spent \$5,000 in the last 10 minutes. A traditional database cannot aggregate 30 days of history in 50ms to feed the model. **The Solution:** Use a Stream Processing framework to maintain a rolling "state" (e.g., total spent in last hour) in memory/fast NoSQL, evaluating rules and calling the ML model instantly on the live stream (Lambda/Kappa Architecture).

Architecture Diagram:



Cloud Architecture Practice Guide for AI Coding Agents

Use this guide to practice building the backend infrastructure for the scenarios we discussed. Copy the **Prompt Templates** below and paste them into your AI coding agent (GitHub Copilot, Cursor, Codeium) to start generating real, deployable code.

1. IoT Edge Batching to Cloud

Tech Stack: Python, SQLite, AWS Boto3 (S3), AWS SQS **AI Prompt Template:**

"Act as a Senior Cloud Engineer. I want to build a simulated IoT pipeline. First, write a Python script that runs locally, generates mock temperature/occupancy data, batches it into an in-memory queue of 60 records, and saves it to a local SQLite database using WAL mode. Second, write a cron-job script that compresses the SQLite file, uploads it to AWS S3, and deletes the local file only upon a 200 OK response."

2. Distributed Ticket Booking (Redis Locks)

Tech Stack: Node.js / Express, Redis (ioredis), PostgreSQL **AI Prompt Template:**

"I am building a high-concurrency ticket booking backend. Create a Node.js Express endpoint `/book-seat`. Connect to a local Redis instance. Implement a Distributed Lock using Redis SETNX with a TTL of 300 seconds for a specific `seat_id`. If the lock is acquired, return success. If the seat is already locked, immediately return an HTTP 409 Conflict. Ensure the code handles edge cases where Redis might fail."

3. FinTech Wallet (Saga Pattern & Idempotency)

Tech Stack: Go or Python (FastAPI), PostgreSQL **AI Prompt Template:**

"Design a basic Saga Orchestrator in Python for a wallet transfer system. Create an endpoint `/transfer`. The code must enforce Idempotency by checking an `idempotency_key` header against a database table before proceeding. Simulate a failure in 'Step 2' (Adding funds to User B) and write the compensating transaction logic to automatically refund User A."

4. E-Commerce CDC Pipeline

Tech Stack: Docker Compose, PostgreSQL, Debezium, Kafka, Elasticsearch **AI Prompt Template:**

"Write a complete `docker-compose.yml` file that sets up a local environment for Change Data Capture (CDC). It needs PostgreSQL, Apache Kafka, Zookeeper/KRaft, Debezium, and Elasticsearch. Then, write a brief README explaining how to configure Debezium to watch a `products` table in Postgres and automatically stream row updates into an Elasticsearch index."

5. Enterprise RAG (LangChain / Vector DB)

Tech Stack: Python, LangChain, OpenAI API, ChromaDB or pgvector **AI Prompt Template:**

"I want to build a secure Enterprise RAG pipeline using LangChain. Write a Python script that takes a sample text document, chunks it using RecursiveCharacterTextSplitter, and embeds it into a local Vector DB. Crucially, show me how to add metadata to these chunks (e.g., `clearance_level: 'tier_3'`). Finally, write the retrieval function that strictly filters by this metadata before passing the context to the LLM."

6. Async Image Generation Queue

Tech Stack: Python (FastAPI), Celery, Redis, AWS S3 **AI Prompt Template:**

"Implement an asynchronous API pattern using FastAPI and Celery. Create a `POST /generate` endpoint that accepts a prompt, instantly returns a `202 Accepted` with a `job_id`, and pushes the task to a Celery worker. Write the Celery worker function to simulate a 10-second sleep (mock GPU generation), generate a dummy text file, upload it to AWS S3 using boto3, and update the task status in a database."

7. AI Context Window Management (Long-Term Memory)

Tech Stack: Python, LangChain/LlamaIndex, Redis, ChromaDB/Pinecone **AI Prompt Template:**

"I am building a conversational AI with infinite memory capabilities that respects strict context window limits. Write a Python script using LangChain or LlamaIndex that implements a 'Windowed Buffer' for the short-term memory (last 5 messages), and a 'VectorStore-backed Memory' for older messages. Show how a new user query automatically combines the short-term buffer with a similarity search of the vector database before constructing the final prompt and calling the LLM."

8. Multi-Tenant RAG & GDPR Data Deletion

Tech Stack: Node.js/Python, PostgreSQL (RLS), Express/FastAPI **AI Prompt Template:**

"Act as a Senior Database Engineer. I am building a multi-tenant B2B SaaS platform. First, write a PostgreSQL schema that creates a `users` and `orders` table. Implement Row-Level Security (RLS) policies so that queries can only access rows where the `tenant_id` matches the current session variable. Second, write a backend API function (using Node.js or Python) that securely sets this session variable based on an incoming JWT token before executing a `SELECT` query. Finally, write a worker script that processes a GDPR 'Right to be Forgotten' event by hard-deleting a user's PII across these tables."

9. Data Lakehouse ETL (Medallion Architecture)

Tech Stack: Python, PySpark, Apache Airflow **AI Prompt Template:**

"Write an Apache Airflow DAG in Python that simulates a Medallion Architecture ETL pipeline. The DAG should have three steps using PySpark: 1) A 'Bronze' task that reads raw JSON log files from a local directory (simulating S3). 2) A 'Silver' task that cleans the data by dropping null values, standardizing timestamp formats, and saving the output as compressed Parquet files. 3) A 'Gold' task that reads the Parquet files, performs a `GROUP BY` aggregation (e.g., counting daily active users), and saves the final analytical table."

10. Real-Time Stream Processing (Fraud Detection)

Tech Stack: Python, Apache Kafka, Apache Flink (or Faust), Redis **AI Prompt Template:**

"Act as a Data Engineer. I want to build a real-time stream processing pipeline for fraud detection. Write a Python producer script that pushes mock credit card transactions (JSON with `user_id`, `amount`, `timestamp`) into a Kafka topic. Then, write a stream processing script (using Faust or PyFlink) that consumes this topic, maintains a rolling sum of the total amount spent by each `user_id` over the last 10 minutes in memory or Redis, and prints a 'FRAUD ALERT' message if the 10-minute rolling sum exceeds \$5000."